

Chapter 5

Two-dimensional array (matrix)

1. Bidimensional array

A two-dimensional array in computer science organizes data into rows and columns, resembling a grid or matrix. It differs from a one-dimensional array by allowing elements to be accessed using two indices: one for the row and one for the column.

1.1. Representation

The matrix is represented in memory by a sequence of adjacent cells. A cell cannot be removed or added to the matrix after its creation (static). The following figure represents a matrix with 4 rows and 5 columns of real numbers.

	0	1	2	3	4
0	15	7	-3	0	9
1	6	12	4	33	85
2	2	-8	17	28	-52
3	14	42	36	49	-12

1.2. Declaration

matrixName: **Array**[Rows][Columns] **of** elementType;

matrixName`: The identifier given to the matrix (variable name).

Rows`: Number of rows.

Columns`: Number of columns.

elementType`: The type of elements. It can be any type, such as integer, float, ...

The number of elements is the product of the number of rows and the number of columns.

Example :

const R=100 ;

const C=100;

M : **Array** [R][C] **of** real ;

mat1,mat2 : **array**[30][20] of integer;

1.3. Accessing elements of a matrix

To access a single element of the matrix, we use the matrix name with an index inside two brackets `[` and `]` specifying the row number, and another index inside two brackets `[` and `]` specifying the column number.

To access the element in the first row and first column of matrix `mat`, we use `mat[0][0]`.

Syntax:

```
array_name [i][j]
```

1.4. Reading a matrix

To fill the matrix `M[Rows][Columns]`, we fill in `Rows` rows. Since each row is a one-dimensional array with `Columns` elements.

```
For ( j ← 0 to C-1 ) do  
  read(M[0][j])  
end for
```

```
For ( j ← 0 to C-1 ) do  
  read(M[1][j])  
endfor
```

```
for (j ← 0 to C-1 ) do  
  read(M[L-1][j]) ;  
endFor
```

So, we can fill all the matrix using double loop

```
For ( i ← 0 to L -1) do  
  For ( j ← 0 to C-1) do  
    read(M[i][j]);  
  endfor;  
enfor;
```

1.5. Displaying a Matrix

Similar to reading, the `write` statement is repeated.

```
for (i ← 0 to L-1) do  
  for (j ← 0 to C-1 ) faire  
    write (M[i][j])  
  endFor  
endFor
```

Example: Sum of elements of a matrix

```
algorithm sum_mat ;  
const n=4, m=5;  
var      mat : array[n-1][m-1] of integer ;  
i, j, sum : integer ;  
begin  
  for (i ← 0 to n -1) do begin  
    sum ← 0 ;  
    for (j ← 0 to m-1) do  
      sum ← sum + mat[i][j] ;  
    endFor;  
    write (sum);  
  endfor;  
end
```